

# Logistic Regression and Random Forest to Determine Credit Card Default

Travis Gubbe

June 27, 2026

In this session, I examine the Credit Card Clients data set found on the UCI Machine Learning Repository website to determine if a person will default on their credit card using logistic regression and random forest.

## About the Data Set

The data set can be found on the UCI Machine Learning Repository site at the following link: <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>.

The data contains 24 variables and a total of 30,000 individual instances. The variables are:

- Default Payment (0 = No, 1 = Yes)
- Amount of Given Credit
- Gender (1 = Male, 2 = Female)
- Education (1 = Graduate School, 2 = University, 3 = High School, 4 = Other, 5 = Unknown)
- Marital Status (1 = Married, 2 = Single, 3 = Others)
- Age
- History of Past Payment from April 2005 to September 2005 where -2=no consumption, -1=pay duly, 0=the use of revolving credit, 1=payment delay for one month, 2=payment delay for two months, ... 9=payment delay for nine months and above
- Amount of Bill Statement from April 2005 to September 2005
- Amount of Previous Payment April 2005 to September 2005

There is a separate variable for each past payment, bill statement, and previous payment from April to September.

## Exploratory Analysis

First, the data set is loaded into a data frame named “credit”. The data frame is also viewed to see the columns and class types.

```
data <- read.csv("~/R/datasets/Credit_Card.csv")
credit = subset(data, select = c("Limit_Bal", "Sex", "Education", "Marriage", "Age", "Pay_Sep",
                                "Pay_Aug", "Pay_July", "Pay_June", "Pay_May", "Pay_April",
                                "Bill_Amt_Sep", "Bill_Amt_Aug", "Bill_Amt_July",
                                "Bill_Amt_June", "Bill_Amt_May", "Bill_Amt_April",
                                "Pay_Amt_Sep", "Pay_Amt_Aug", "Pay_Amt_July", "Pay_Amt_June",
                                "Pay_Amt_May", "Pay_Amt_April", "Default"))

#Inspect the data frame
head(credit)
```

```
##   Limit_Bal Sex Education Marriage Age Pay_Sep Pay_Aug Pay_July Pay_June
## 1    20000   2         2         1  24        2        2        -1        -1
## 2   120000   2         2         2  26       -1        2         0         0
```

```
## 3      90000    2      2      2 34      0      0      0      0
## 4      50000    2      2      1 37      0      0      0      0
## 5      50000    1      2      1 57     -1      0     -1      0
## 6      50000    1      1      2 37      0      0      0      0
##      Pay_May Pay_April Bill_Amt_Sep Bill_Amt_Aug Bill_Amt_July Bill_Amt_June
## 1      -2      -2      3913      3102      689      0
## 2       0       2      2682      1725      2682      3272
## 3       0       0      29239      14027      13559      14331
## 4       0       0      46990      48233      49291      28314
## 5       0       0       8617       5670      35835      20940
## 6       0       0      64400      57069      57608      19394
##      Bill_Amt_May Bill_Amt_April Pay_Amt_Sep Pay_Amt_Aug Pay_Amt_July Pay_Amt_June
## 1          0          0          0          689          0          0
## 2       3455       3261          0       1000       1000       1000
## 3      14948      15549       1518       1500       1000       1000
## 4      28959      29547       2000       2019       1200       1100
## 5      19146      19131       2000      36681      10000       9000
## 6      19619      20024       2500       1815        657       1000
##      Pay_Amt_May Pay_Amt_April Default
## 1          0          0          1
## 2          0       2000          1
## 3       1000       5000          0
## 4       1069       1000          0
## 5         689        679          0
## 6       1000        800          0
```

```
#Inspect the classes of the data frame
sapply(credit, class)
```

```
##      Limit_Bal      Sex      Education      Marriage      Age
##      "integer"    "integer"    "integer"    "integer"    "integer"
##      Pay_Sep      Pay_Aug      Pay_July      Pay_June      Pay_May
##      "integer"    "integer"    "integer"    "integer"    "integer"
##      Pay_April    Bill_Amt_Sep    Bill_Amt_Aug    Bill_Amt_July    Bill_Amt_June
##      "integer"    "integer"    "integer"    "integer"    "integer"
##      Bill_Amt_May    Bill_Amt_April    Pay_Amt_Sep    Pay_Amt_Aug    Pay_Amt_July
##      "integer"    "integer"    "integer"    "integer"    "integer"
##      Pay_Amt_June    Pay_Amt_May    Pay_Amt_April    Default
##      "integer"    "integer"    "integer"    "integer"
```

```
attach(credit)
```

Next, I want to see the amount of missing values and duplicates in the data frame.

```
sum(is.na(credit))
```

```
## [1] 0
```

```
duplicates <- credit%>%duplicated()
duplicates_amount <- duplicates%>%(table)
duplicates_amount
```

```
## .
## FALSE  TRUE
## 29965   35
```

Since there are 35 duplicates in the data, the data frame is filtered to remove the duplicates.

```
credit <- credit%>%distinct()
#Displays how many duplicates are present in the updated data frame.
duplicates_counts_unique <- credit%>%duplicated()%>%table()
duplicates_counts_unique
```

```
## .
## FALSE
## 29965
```

Next, the factor variables are converted from their numeric values to their actual names. This is done on a copy of the credit data frame.

```
credit1 <- data.frame(credit)
head(credit1)
```

```
##   Limit_Bal Sex Education Marriage Age Pay_Sep Pay_Aug Pay_July Pay_June
## 1    20000  2      2      1    24      2      2      -1      -1
## 2   120000  2      2      2    26     -1      2      0      0
## 3    90000  2      2      2    34      0      0      0      0
## 4    50000  2      2      1    37      0      0      0      0
## 5    50000  1      2      1    57     -1      0     -1      0
## 6    50000  1      1      2    37      0      0      0      0
##   Pay_May Pay_April Bill_Amt_Sep Bill_Amt_Aug Bill_Amt_July Bill_Amt_June
## 1      -2      -2      3913      3102      689      0
## 2       0       2      2682      1725      2682      3272
## 3       0       0     29239     14027     13559     14331
## 4       0       0     46990     48233     49291     28314
## 5       0       0      8617      5670     35835     20940
## 6       0       0     64400     57069     57608     19394
##   Bill_Amt_May Bill_Amt_April Pay_Amt_Sep Pay_Amt_Aug Pay_Amt_July Pay_Amt_June
## 1          0          0          0          689          0          0
## 2       3455       3261          0       1000       1000       1000
## 3      14948      15549      1518       1500       1000       1000
## 4      28959      29547      2000       2019       1200       1100
## 5      19146      19131      2000      36681      10000       9000
## 6      19619      20024      2500       1815        657       1000
##   Pay_Amt_May Pay_Amt_April Default
## 1          0          0          1
## 2          0       2000          1
## 3       1000       5000          0
## 4       1069       1000          0
## 5         689        679          0
## 6       1000        800          0
```

```
#Rename factor variables to their appropriate settings
credit1$Sex[credit$Sex %in% "1"] = "Male"
credit1$Sex[credit$Sex %in% "2"] = "Female"

credit1$Education[credit$Education %in% "1"] = "Grad School"
credit1$Education[credit$Education %in% "2"] = "College"
credit1$Education[credit$Education %in% "3"] = "High School"
credit1$Education[credit$Education %in% "4"] = "Other"
credit1$Education[credit$Education %in% "5"] = "Unknown"

credit1$Marriage[credit$Marriage %in% "0"] = "Unknown"
```

```

credit1$Marriage[credit$Marriage %in% "1"] = "Married"
credit1$Marriage[credit$Marriage %in% "2"] = "Single"
credit1$Marriage[credit$Marriage %in% "3"] = "Other"

credit1$Default[credit$Default %in% "0"] = "No"
credit1$Default[credit$Default %in% "1"] = "Yes"
#See the change in the variable names
head(credit1)

```

```

##   Limit_Bal   Sex   Education Marriage Age Pay_Sep Pay_Aug Pay_July Pay_June
## 1    20000 Female   College   Married  24      2      2      -1      -1
## 2   120000 Female   College   Single   26     -1      2       0       0
## 3    90000 Female   College   Single   34      0      0       0       0
## 4    50000 Female   College   Married  37      0      0       0       0
## 5    50000  Male   College   Married  57     -1      0      -1       0
## 6    50000  Male Grad School   Single  37      0      0       0       0
##   Pay_May Pay_April Bill_Amt_Sep Bill_Amt_Aug Bill_Amt_July Bill_Amt_June
## 1      -2        -2        3913        3102          689           0
## 2       0         2        2682        1725         2682        3272
## 3       0         0       29239       14027       13559       14331
## 4       0         0       46990       48233       49291       28314
## 5       0         0        8617        5670       35835       20940
## 6       0         0       64400       57069       57608       19394
##   Bill_Amt_May Bill_Amt_April Pay_Amt_Sep Pay_Amt_Aug Pay_Amt_July Pay_Amt_June
## 1           0           0           0           689           0           0
## 2        3455          3261           0          1000         1000         1000
## 3       14948       15549        1518          1500         1000         1000
## 4       28959       29547        2000          2019         1200         1100
## 5       19146       19131        2000       36681       10000         9000
## 6       19619       20024        2500        1815          657         1000
##   Pay_Amt_May Pay_Amt_April Default
## 1           0           0      Yes
## 2           0        2000      Yes
## 3        1000        5000       No
## 4        1069        1000       No
## 5         689         679       No
## 6        1000         800       No

```

Next, exploratory tables are made to view the distribution of the data set.

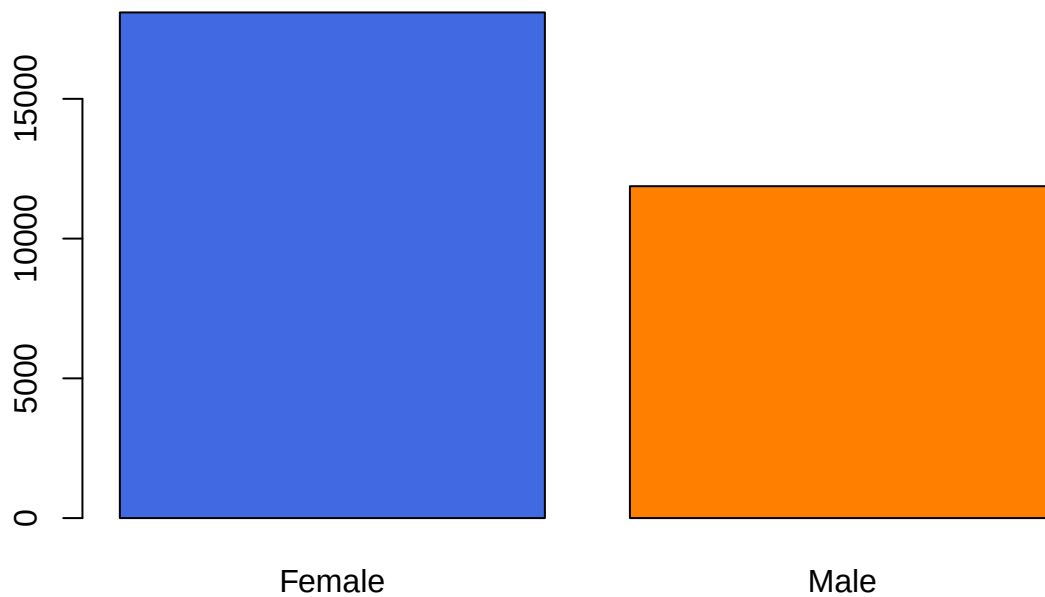
## Exploratory Analysis

Next, bar plots and distribution tables are created to see the proportion of the variables. This is done to see if the data is normally distributed. If the data is not normally distributed, it's advantageous to see how the data is skewed.

```

#View the bar plots for the amount for each categorical variable
counts_Sex <- table(credit1$Sex)
barplot(counts_Sex, col = c("royalblue", "darkorange1"))

```



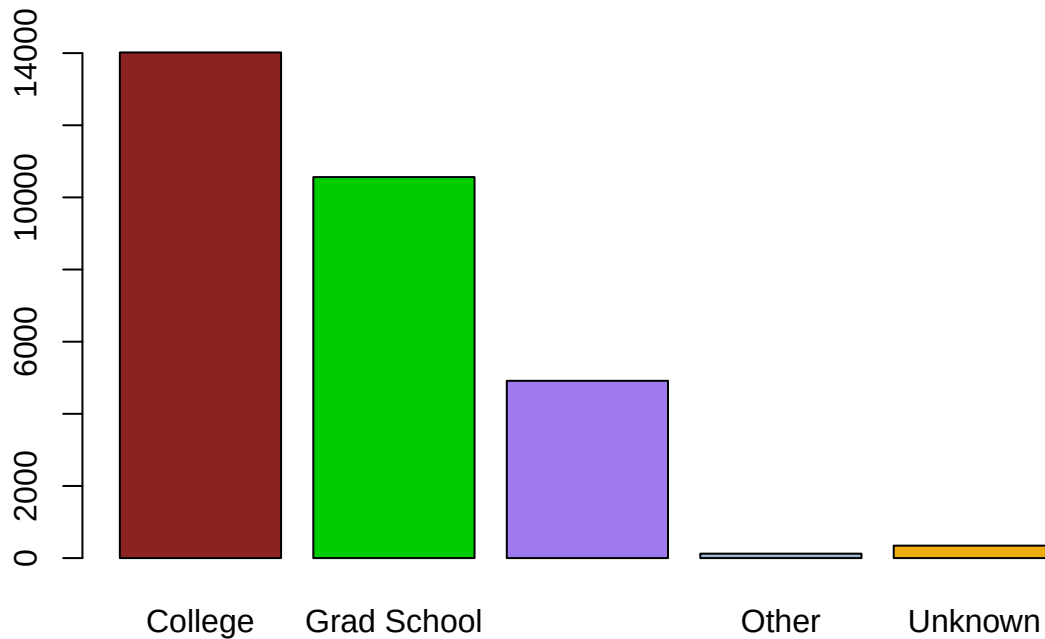
```
#Basic table view of the amount of males and females  
table(credit1$Sex)
```

```
##  
## Female    Male  
## 18091    11874
```

```
#Proportion of each gender in table  
prop.table(counts_Sex)
```

```
##  
##      Female      Male  
## 0.6037377 0.3962623
```

```
counts_Education <- table(credit1$Education)  
barplot(counts_Education, col = c("brown4", "green3", "mediumpurple2", "slategray3",  
                                  "darkgoldenrod2"))
```



```
table(credit1$Education)
```

```
##
##      College Grad School High School      Other      Unknown
##      14019      10563      4915          123          345
```

```
#Proportion of each education level in table
prop.table(counts_Education)
```

```
##
##      College Grad School High School      Other      Unknown
## 0.467845820 0.352511263 0.164024695 0.004104789 0.011513432
```

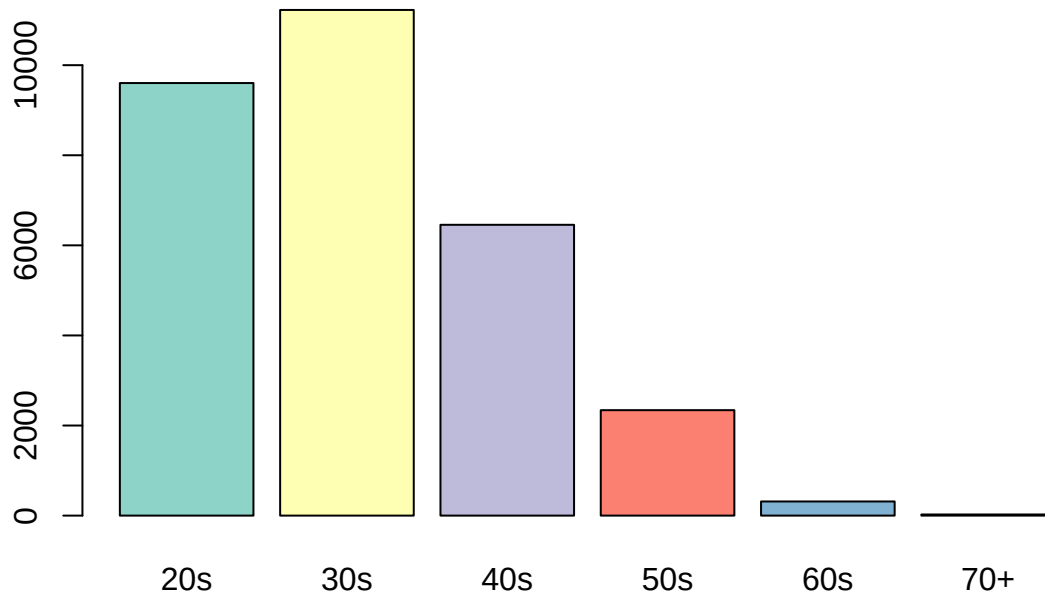
```
#Age groups divided into decades, then proportion table of each age group.
credit1$Age_Group <- cut(credit$Age,
                        breaks = c(20, 30, 40, 50, 60, 70, 80),
                        labels = c("20s", "30s", "40s", "50s", "60s", "70+"),
                        right = FALSE)
```

```
counts_Age <- table(credit1$Age_Group)
# Check distribution
prop.table(counts_Age)
```

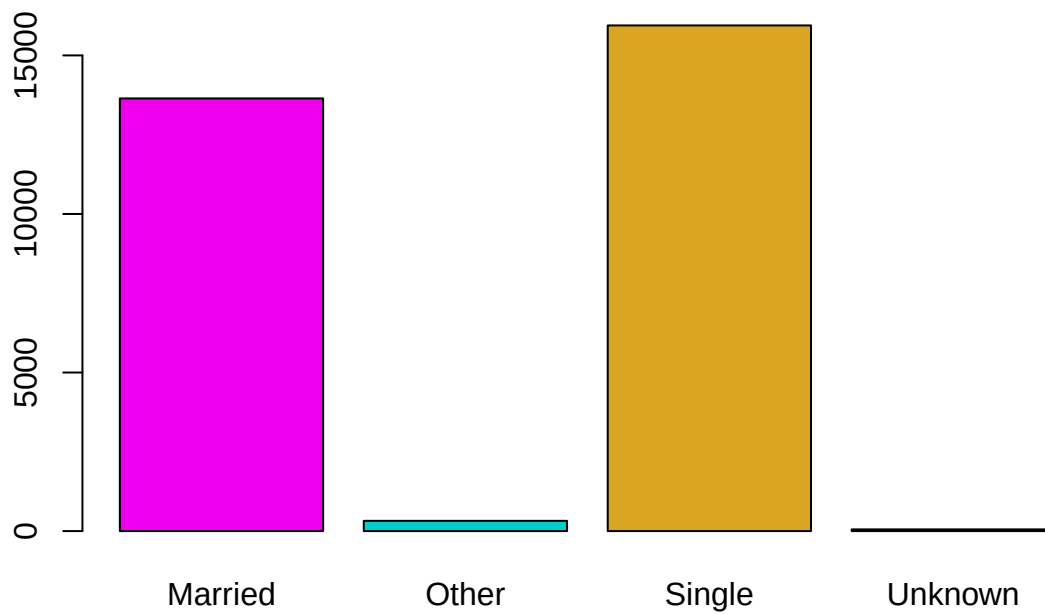
```
##
##      20s      30s      40s      50s      60s      70+
## 0.3204738862 0.3746370766 0.2154513599 0.0781244786 0.0104788920 0.0008343067
```

```
#Bar plot of age groups with custom colors
colors <- brewer.pal(6, "Set3")
```

```
barplot(counts_Age, col = colors)
```



```
#Counts for each marriage status.  
counts_Marriage <- table(credit1$Marriage)  
barplot(counts_Marriage, col = c("magenta2", "Cyan3", "goldenrod"))
```



```
table(credit$Marriage)
```

```
##
##      0      1      2      3
##  54 13643 15945   323
```

```
#Proportion of each marriage status in table
```

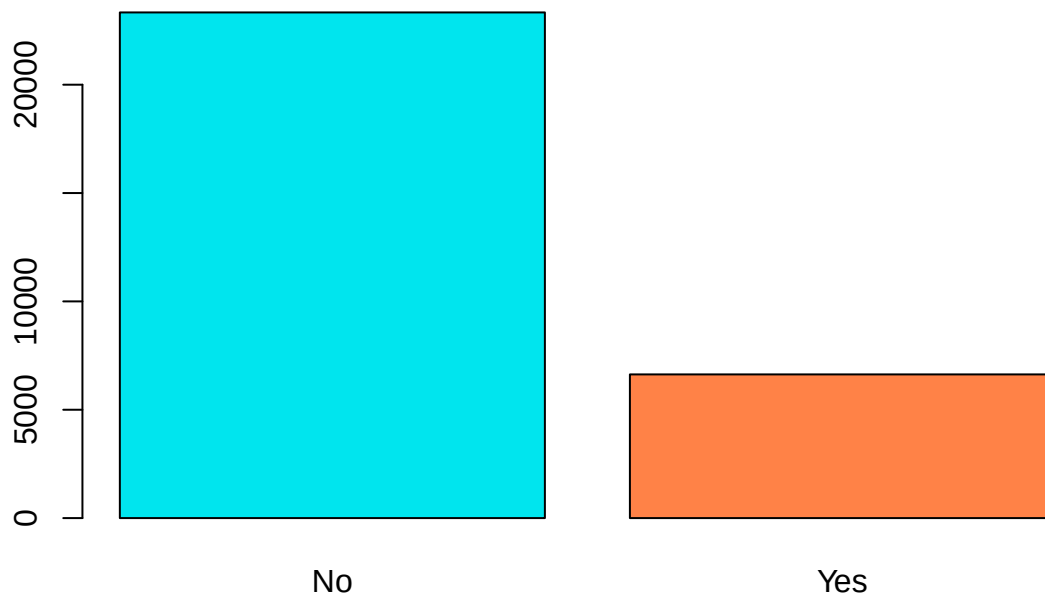
```
prop.table(counts_Marriage)
```

```
##
##      Married      Other      Single      Unknown
## 0.455297847 0.010779242 0.532120808 0.001802102
```

```
counts_Default <- table(credit1$Default)
```

```
barplot(counts_Default, col = c("turquoise2", "sienna1"))
```





```
table(credit$Default)
```

```
##
##      0      1
## 23335  6630
```

```
prop.table(counts_Default)
```

```
##
##      No      Yes
## 0.7787419 0.2212581
```

```
table.default_gender <- table(credit1$Default, credit$Sex)
prop.table(table.default_gender, 2)
```

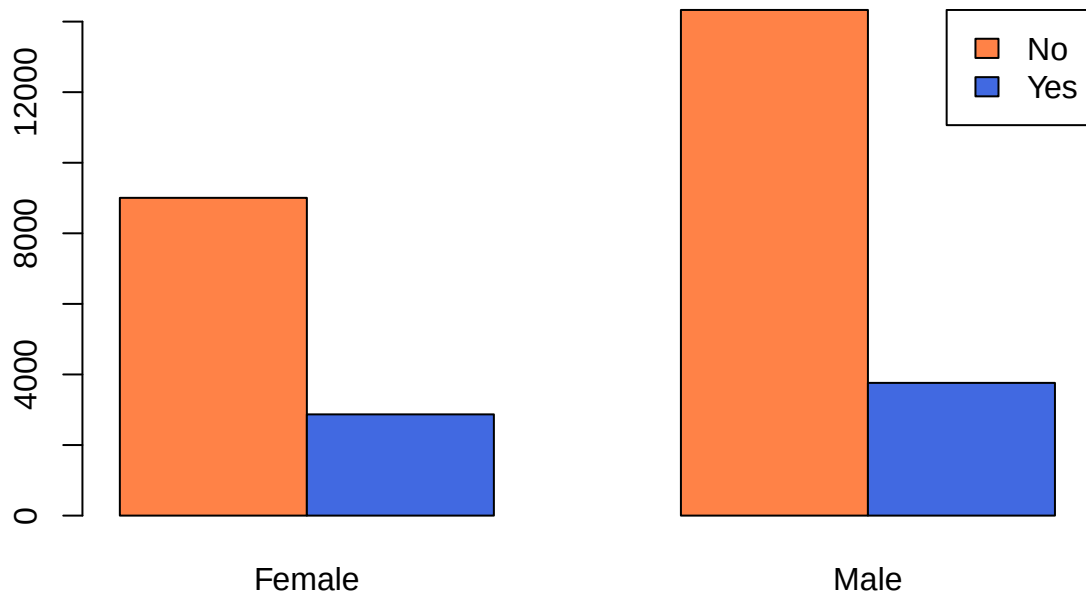
```
##
##           1           2
##  No  0.7583797 0.7921066
##  Yes 0.2416203 0.2078934
```

```
prop.table(table.default_gender, 1)
```

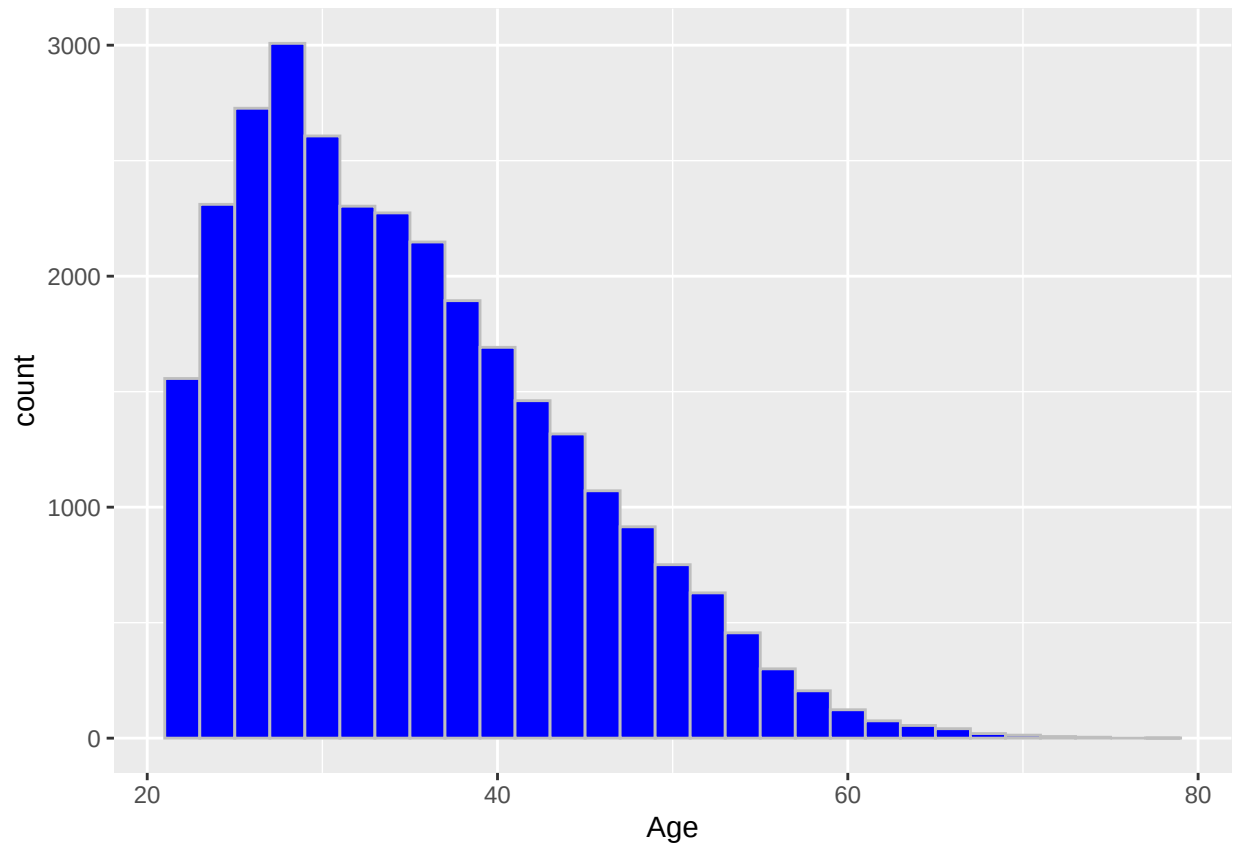
```
##
##           1           2
##  No  0.385901 0.614099
##  Yes 0.432730 0.567270
```

```
barplot(table.default_gender, col = c("sienna1", "royalblue"), beside = T,
        names.arg = c("Female", "Male"))
```

```
legend("topright", legend = c("No", "Yes"), fill = c("sienna1", "royalblue"))
```

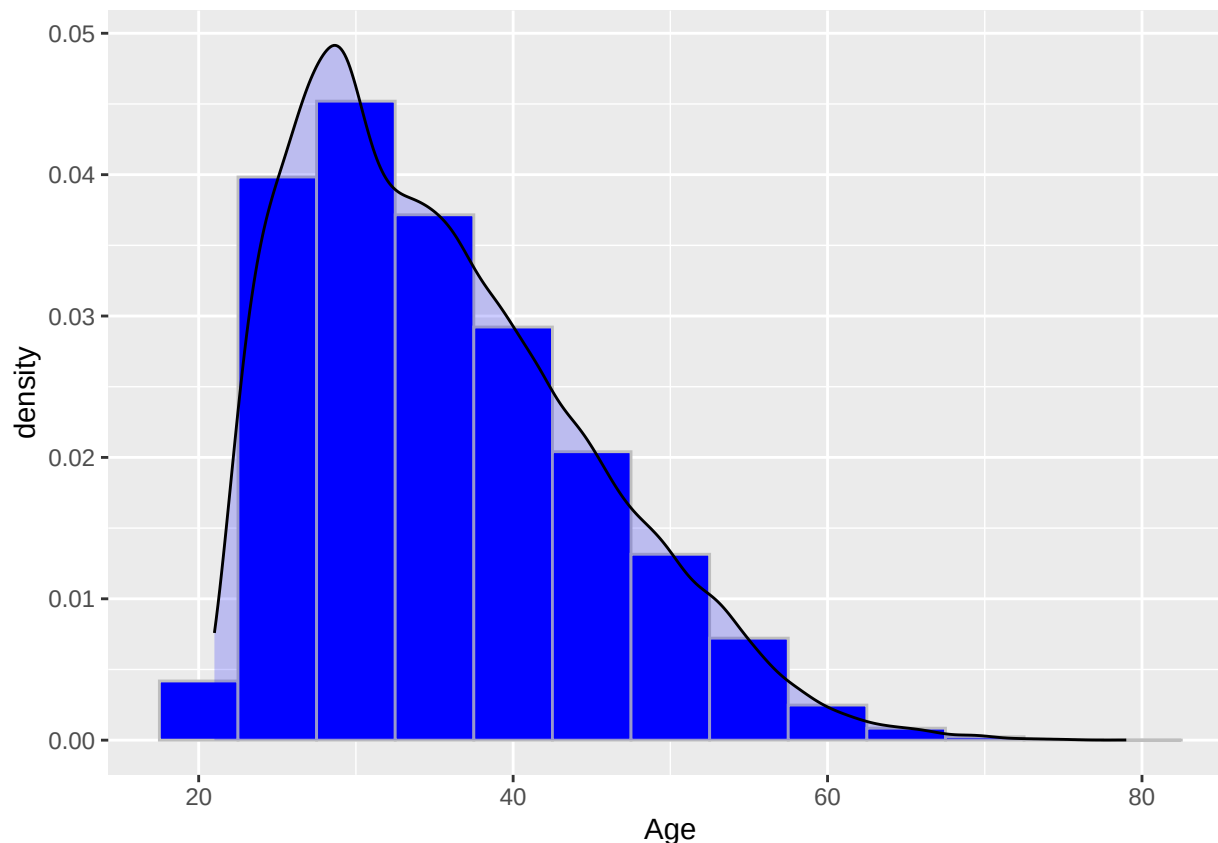


```
ggplot(data = credit, aes(x = Age)) + geom_histogram(fill = "Blue", col = "Grey", bins = 30)
```



```
ggplot(data = credit, aes(x = Age)) + geom_histogram(aes(y = ..density..), fill = "Blue", col = "Grey",
```

```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.  
## i Please use `after_stat(density)` instead.  
## This warning is displayed once every 8 hours.  
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was  
## generated.
```



```
mean(credit1$Age)
```

```
## [1] 35.48797
```

Looking at the created charts and tables, the data has more females than males. In addition, the age group distribution is skewed to the right, meaning the data is represented by younger participants.

Another method for exploratory analysis is to investigate the correlation of the numeric variables. The correlation of numeric variables can help with model selection later on, as high correlation can indicate multicollinearity among variables, which can make it difficult for a model to separate and distinguish the importance of individual variables in the final model selection.

```
eda_numeric_only <- credit1[, c("Limit_Bal", "Age",  
                                "Bill_Amt_Sep", "Bill_Amt_Aug", "Bill_Amt_July", "Bill_Amt_June", "Bill_Amt_May",  
                                "Bill_Amt_April", "Pay_Amt_Sep", "Pay_Amt_Aug", "Pay_Amt_July", "Pay_Amt_June",  
                                "Pay_Amt_May", "Pay_Amt_April")]
```

```
# 2. Compute the Pearson correlation matrix safely
```

```
clean_eda_cor <- cor(eda_numeric_only, use = "complete.obs")
```

```
# 3. Print the perfectly aligned results rounded to 2 decimal places
```

```
print(round(clean_eda_cor, 2))
```

```
##          Limit_Bal  Age Bill_Amt_Sep Bill_Amt_Aug Bill_Amt_July
## Limit_Bal      1.00 0.14          0.29          0.28          0.28
## Age            0.14 1.00          0.06          0.05          0.05
## Bill_Amt_Sep    0.29 0.06          1.00          0.95          0.89
## Bill_Amt_Aug    0.28 0.05          0.95          1.00          0.93
```

```
## Bill_Amt_July      0.28 0.05      0.89      0.93      1.00
## Bill_Amt_June      0.29 0.05      0.86      0.89      0.92
## Bill_Amt_May       0.30 0.05      0.83      0.86      0.88
## Bill_Amt_April     0.29 0.05      0.80      0.83      0.85
## Pay_Amt_Sep        0.20 0.03      0.14      0.28      0.24
## Pay_Amt_Aug        0.18 0.02      0.10      0.10      0.32
## Pay_Amt_July       0.21 0.03      0.16      0.15      0.13
## Pay_Amt_June       0.20 0.02      0.16      0.15      0.14
## Pay_Amt_May        0.22 0.02      0.17      0.16      0.18
## Pay_Amt_April     0.22 0.02      0.18      0.17      0.18
##
##      Bill_Amt_June Bill_Amt_May Bill_Amt_April Pay_Amt_Sep
## Limit_Bal      0.29      0.30      0.29      0.20
## Age            0.05      0.05      0.05      0.03
## Bill_Amt_Sep    0.86      0.83      0.80      0.14
## Bill_Amt_Aug    0.89      0.86      0.83      0.28
## Bill_Amt_July   0.92      0.88      0.85      0.24
## Bill_Amt_June   1.00      0.94      0.90      0.23
## Bill_Amt_May    0.94      1.00      0.95      0.22
## Bill_Amt_April  0.90      0.95      1.00      0.20
## Pay_Amt_Sep     0.23      0.22      0.20      1.00
## Pay_Amt_Aug     0.21      0.18      0.17      0.29
## Pay_Amt_July    0.30      0.25      0.23      0.25
## Pay_Amt_June    0.13      0.29      0.25      0.20
## Pay_Amt_May     0.16      0.14      0.31      0.15
## Pay_Amt_April   0.18      0.16      0.12      0.19
##
##      Pay_Amt_Aug Pay_Amt_July Pay_Amt_June Pay_Amt_May Pay_Amt_April
## Limit_Bal      0.18      0.21      0.20      0.22      0.22
## Age            0.02      0.03      0.02      0.02      0.02
## Bill_Amt_Sep    0.10      0.16      0.16      0.17      0.18
## Bill_Amt_Aug    0.10      0.15      0.15      0.16      0.17
## Bill_Amt_July   0.32      0.13      0.14      0.18      0.18
## Bill_Amt_June   0.21      0.30      0.13      0.16      0.18
## Bill_Amt_May    0.18      0.25      0.29      0.14      0.16
## Bill_Amt_April  0.17      0.23      0.25      0.31      0.12
## Pay_Amt_Sep     0.29      0.25      0.20      0.15      0.19
## Pay_Amt_Aug     1.00      0.24      0.18      0.18      0.16
## Pay_Amt_July    0.24      1.00      0.22      0.16      0.16
## Pay_Amt_June    0.18      0.22      1.00      0.15      0.16
## Pay_Amt_May     0.18      0.16      0.15      1.00      0.15
## Pay_Amt_April   0.16      0.16      0.16      0.15      1.00
```

From the results, there are a few instances of high correlation between various numeric variables, particularly among variables with similar measurements (Bill\_Amt\_April and Bill\_Amt\_May). This will play a factor later in the analysis, particularly for the logistic regression model's regularization techniques.

## Train and Test Sets

Before creating prediction models, training and testing data sets are created. A training data set is a subset of examples used to train the model, while the testing data set is a subset used to test the training model.

```
#Initializes number generator.
```

```
set.seed(123)
```

```
#New sample created for the training and testing data sets. The data is split with 75% in training and .
```

```
sample_split <- sample(c(TRUE, FALSE), nrow(credit), replace = TRUE, prob = c(0.75, 0.25))
```

```
train_set <- credit[sample_split, ]
```

```
test_set <- credit[!sample_split, ]

X_train <- model.matrix(Default ~ ., data = train_set)[, -1]
y_train <- as.numeric(train_set$Default)

# 2. Prepare the TESTING data matrices
X_test <- model.matrix(Default ~ ., data = test_set)[, -1]
y_test <- as.numeric(test_set$Default)
```

## Logistic Regression

First, logistic regression is done to find the probability of default for an individual. Logistic regression models the probability that a response variable (Y) belongs to a particular category. This method uses maximum likelihood to fit the model in the range between 0 and 1.

Logistic regression is a classification method great for a yes/no response. A number closer to 1 represents “Yes”, while a number closer to 0 represents “No”.

Before the creation of the model, An elastic net logistic regression model is first run as a means of feature selection and to prevent overfitting in the model. Elastic net is a hybrid version of the LASSO and ridge regularization methods, performing both coefficient shrinkage and feature selection which is appropriate in this case due to the high correlation among multiple variables. The elastic net has more flexibility than the LASSO regularization and is more robust than the Ridge regularization, selecting and dropping entire an entire group of correlated groups.

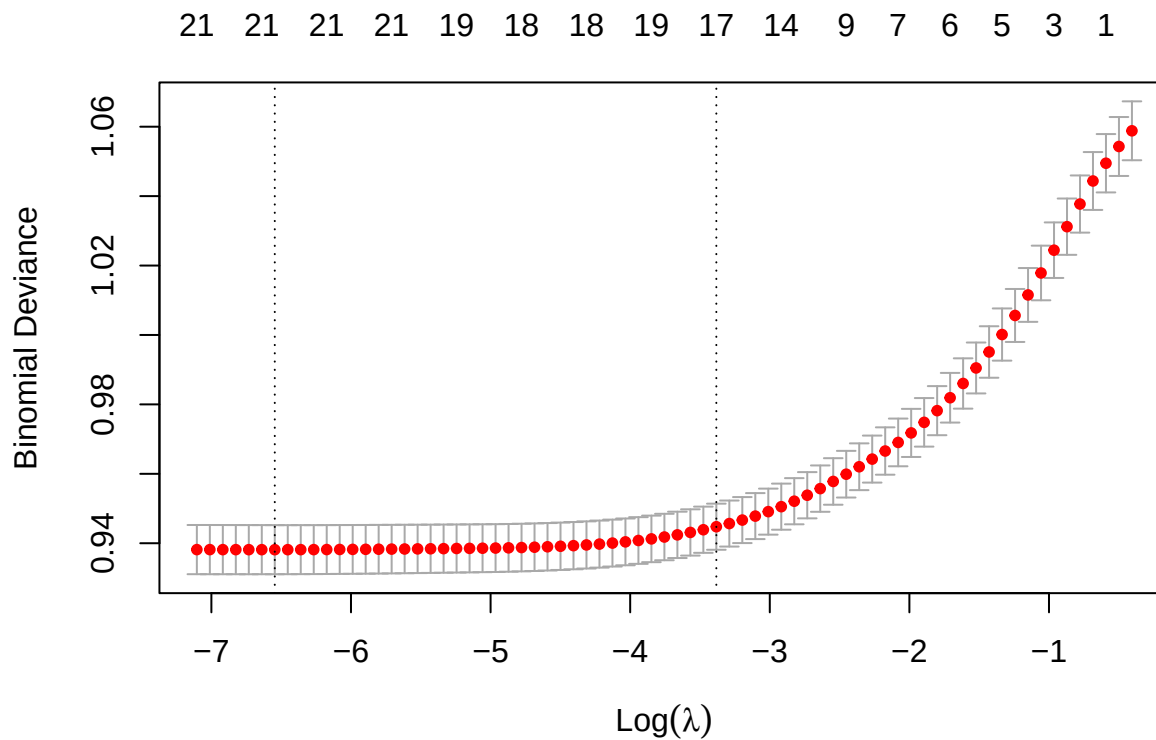
From the glmnet package, any alpha value between 0 and 1 is classified as elastic net. A for loop was created to determine the best alpha value among when separating by between three alpha values (0.2, 0.5, and 0.8).

```
library(glmnet)
#Elastic net for the data set
#Determine best alpha value
alphas <- c(0.2, 0.5, 0.8)
for (a in alphas) {
  set.seed(123)
  cv_model <- cv.glmnet(X_train, y_train, family = "binomial", alpha = a)
  cat("Alpha:", a, " Minimum Deviance:", min(cv_model$cvm), "\n")
}

## Alpha: 0.2 Minimum Deviance: 0.9382583
## Alpha: 0.5 Minimum Deviance: 0.9382775
## Alpha: 0.8 Minimum Deviance: 0.938284

cv_en <- cv.glmnet(X_train, y_train, family = "binomial", alpha = 0.2, standardize = TRUE)

plot(cv_en)
```



```
en_coefs <- coef(cv_en, s = "lambda.1se")
print(en_coefs)
```

```
## 24 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept)  -1.063969e+00
## Limit_Bal    -6.226294e-07
## Sex          .
## Education    -3.876596e-03
## Marriage      -9.217191e-02
## Age          2.371693e-03
## Pay_Sep      4.390245e-01
## Pay_Aug      1.055434e-01
## Pay_July     5.965425e-02
## Pay_June     2.679787e-02
## Pay_May      3.079567e-02
## Pay_April    1.334030e-02
## Bill_Amt_Sep -7.911601e-07
## Bill_Amt_Aug -1.380020e-07
## Bill_Amt_July .
## Bill_Amt_June .
## Bill_Amt_May .
## Bill_Amt_April .
## Pay_Amt_Sep  -4.444265e-06
## Pay_Amt_Aug  -2.242520e-06
## Pay_Amt_July -6.183747e-07
```

```
## Pay_Amt_June    -1.317573e-06
## Pay_Amt_May     -4.905997e-07
## Pay_Amt_April    .
```

From the results, the optimal value is 0.2 since it has the lowest minimum deviance. The elastic net regularization is then run, which reduces the model to 17 variables. logistic regression model is created below using the 17 variables to start:

```
fit_glm_cv <- glm(Default ~ Limit_Bal+Marriage+Age+Pay_Sep+Pay_Aug+Pay_July+
  Pay_June+Bill_Amt_Sep+Pay_Amt_Sep+Pay_Amt_Aug+
  Pay_Amt_July+Pay_Amt_June+Pay_Amt_May,
  data = train_set, family = binomial())
```

```
summary(fit_glm_cv)
```

```
##
## Call:
## glm(formula = Default ~ Limit_Bal + Marriage + Age + Pay_Sep +
##      Pay_Aug + Pay_July + Pay_June + Bill_Amt_Sep + Pay_Amt_Sep +
##      Pay_Amt_Aug + Pay_Amt_July + Pay_Amt_June + Pay_Amt_May,
##      family = binomial(), data = train_set)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.039e+00  1.095e-01  -9.491  < 2e-16 ***
## Limit_Bal    -5.282e-07  1.726e-07  -3.060  0.002211 **
## Marriage      -1.704e-01  3.604e-02  -4.728  2.27e-06 ***
## Age           6.365e-03  1.987e-03   3.204  0.001358 **
## Pay_Sep       5.768e-01  2.041e-02  28.262  < 2e-16 ***
## Pay_Aug       8.705e-02  2.309e-02   3.770  0.000164 ***
## Pay_July      6.713e-02  2.560e-02   2.622  0.008735 **
## Pay_June      6.018e-02  2.269e-02   2.652  0.008003 **
## Bill_Amt_Sep -1.485e-06  3.024e-07  -4.912  9.03e-07 ***
## Pay_Amt_Sep  -1.044e-05  2.329e-06  -4.480  7.46e-06 ***
## Pay_Amt_Aug  -1.004e-05  2.204e-06  -4.553  5.28e-06 ***
## Pay_Amt_July -2.328e-06  1.672e-06  -1.393  0.163708
## Pay_Amt_June -4.033e-06  1.760e-06  -2.291  0.021966 *
## Pay_Amt_May  -2.386e-06  1.581e-06  -1.509  0.131195
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 23800  on 22476  degrees of freedom
## Residual deviance: 21069  on 22463  degrees of freedom
## AIC: 21097
##
## Number of Fisher Scoring iterations: 6
```

```
stepAIC(fit_glm_cv)
```

```
## Start:  AIC=21097.12
## Default ~ Limit_Bal + Marriage + Age + Pay_Sep + Pay_Aug + Pay_July +
##      Pay_June + Bill_Amt_Sep + Pay_Amt_Sep + Pay_Amt_Aug + Pay_Amt_July +
##      Pay_Amt_June + Pay_Amt_May
##
```



```
##           Df Deviance   AIC
## <none>           21069 21097
## - Pay_Amt_July  1     21071 21097
## - Pay_Amt_May   1     21072 21098
## - Pay_Amt_June  1     21075 21101
## - Pay_July      1     21076 21102
## - Pay_June      1     21076 21102
## - Limit_Bal     1     21079 21105
## - Age           1     21079 21105
## - Pay_Aug       1     21083 21109
## - Marriage       1     21092 21118
## - Bill_Amt_Sep  1     21094 21120
## - Pay_Amt_Sep   1     21097 21123
## - Pay_Amt_Aug   1     21099 21125
## - Pay_Sep       1     21860 21886
```

```
##
## Call: glm(formula = Default ~ Limit_Bal + Marriage + Age + Pay_Sep +
##       Pay_Aug + Pay_July + Pay_June + Bill_Amt_Sep + Pay_Amt_Sep +
##       Pay_Amt_Aug + Pay_Amt_July + Pay_Amt_June + Pay_Amt_May,
##       family = binomial(), data = train_set)
##
```

```
## Coefficients:
## (Intercept)      Limit_Bal      Marriage      Age      Pay_Sep
## -1.039e+00    -5.282e-07    -1.704e-01    6.365e-03    5.768e-01
##      Pay_Aug      Pay_July      Pay_June  Bill_Amt_Sep  Pay_Amt_Sep
##  8.705e-02    6.713e-02    6.018e-02    -1.485e-06    -1.044e-05
## Pay_Amt_Aug  Pay_Amt_July  Pay_Amt_June  Pay_Amt_May
## -1.004e-05    -2.328e-06    -4.033e-06    -2.386e-06
##
```

```
## Degrees of Freedom: 22476 Total (i.e. Null);  22463 Residual
## Null Deviance:      23800
## Residual Deviance: 21070      AIC: 21100
```

*#VIF of the 2nd logistic regression model. VIF scores appear good.*

```
vif(fit_glm_cv)
```

```
##      Limit_Bal      Marriage      Age      Pay_Sep      Pay_Aug      Pay_July
##      1.407081      1.207696      1.210552      1.514391      2.605634      3.202421
##      Pay_June  Bill_Amt_Sep  Pay_Amt_Sep  Pay_Amt_Aug  Pay_Amt_July  Pay_Amt_June
##      2.425439      1.354999      1.161687      1.149575      1.096311      1.079075
## Pay_Amt_May
##      1.082014
```

```
fit_glm_cv2 <- glm(Default ~ Limit_Bal+Marriage+Age+Pay_Sep+Pay_Aug+Pay_July+
      Pay_June+Bill_Amt_Sep+Bill_Amt_June+Pay_Amt_Sep+Pay_Amt_Aug+Pay_Amt_June,
      data = train_set, family = binomial())
```

```
summary(fit_glm_cv2)
```

```
##
## Call:
## glm(formula = Default ~ Limit_Bal + Marriage + Age + Pay_Sep +
##       Pay_Aug + Pay_July + Pay_June + Bill_Amt_Sep + Bill_Amt_June +
##       Pay_Amt_Sep + Pay_Amt_Aug + Pay_Amt_June, family = binomial(),
##       data = train_set)
```

```

##
## Coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.034e+00  1.095e-01 -9.443  < 2e-16 ***
## Limit_Bal   -6.294e-07  1.721e-07 -3.657  0.000256 ***
## Marriage     -1.719e-01  3.604e-02 -4.769  1.85e-06 ***
## Age         6.377e-03  1.987e-03  3.209  0.001334 **
## Pay_Sep     5.757e-01  2.041e-02 28.205  < 2e-16 ***
## Pay_Aug     8.735e-02  2.309e-02  3.782  0.000155 ***
## Pay_July    6.360e-02  2.563e-02  2.482  0.013064 *
## Pay_June    5.611e-02  2.279e-02  2.462  0.013812 *
## Bill_Amt_Sep -3.087e-06  6.015e-07 -5.132  2.87e-07 ***
## Bill_Amt_June 2.045e-06  6.969e-07  2.934  0.003342 **
## Pay_Amt_Sep  -1.188e-05  2.363e-06 -5.026  5.01e-07 ***
## Pay_Amt_Aug  -1.169e-05  2.269e-06 -5.150  2.61e-07 ***
## Pay_Amt_June -4.274e-06  1.797e-06 -2.379  0.017372 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 23800  on 22476  degrees of freedom
## Residual deviance: 21065  on 22464  degrees of freedom
## AIC: 21091
##
## Number of Fisher Scoring iterations: 6

```

```

stepAIC(fit_glm_cv2)

```

```

## Start:  AIC=21091.29
## Default ~ Limit_Bal + Marriage + Age + Pay_Sep + Pay_Aug + Pay_July +
##      Pay_June + Bill_Amt_Sep + Bill_Amt_June + Pay_Amt_Sep + Pay_Amt_Aug +
##      Pay_Amt_June
##
##           Df Deviance    AIC
## <none>             21065 21091
## - Pay_June         1    21071 21095
## - Pay_July         1    21071 21095
## - Pay_Amt_June     1    21072 21096
## - Bill_Amt_June    1    21074 21098
## - Age              1    21076 21100
## - Limit_Bal        1    21079 21103
## - Pay_Aug          1    21080 21104
## - Marriage          1    21088 21112
## - Bill_Amt_Sep     1    21093 21117
## - Pay_Amt_Sep      1    21101 21125
## - Pay_Amt_Aug      1    21104 21128
## - Pay_Sep          1    21853 21877
##
## Call:  glm(formula = Default ~ Limit_Bal + Marriage + Age + Pay_Sep +
##      Pay_Aug + Pay_July + Pay_June + Bill_Amt_Sep + Bill_Amt_June +
##      Pay_Amt_Sep + Pay_Amt_Aug + Pay_Amt_June, family = binomial(),
##      data = train_set)
##

```

```
## Coefficients:
## (Intercept)      Limit_Bal      Marriage      Age      Pay_Sep
## -1.034e+00    -6.294e-07    -1.719e-01    6.377e-03    5.757e-01
##      Pay_Aug      Pay_July      Pay_June    Bill_Amt_Sep    Bill_Amt_June
##      8.735e-02    6.360e-02    5.611e-02    -3.087e-06    2.045e-06
##      Pay_Amt_Sep    Pay_Amt_Aug    Pay_Amt_June
##      -1.187e-05    -1.169e-05    -4.274e-06
##
## Degrees of Freedom: 22476 Total (i.e. Null); 22464 Residual
## Null Deviance:      23800
## Residual Deviance: 21070      AIC: 21090
```

*#VIF of the 2nd logistic regression model. VIF scores appear good.*

```
vif(fit_glm_cv2)
```

```
##      Limit_Bal      Marriage      Age      Pay_Sep      Pay_Aug
##      1.399953      1.207157      1.211042      1.518304      2.610328
##      Pay_July      Pay_June    Bill_Amt_Sep    Bill_Amt_June    Pay_Amt_Sep
##      3.216057      2.448008      5.355772      5.728086      1.173155
##      Pay_Amt_Aug    Pay_Amt_June
##      1.186303      1.075627
```

*#Elastic net logistic regression model - removed bill\_amt\_aug (high VIF)*

*#Removed pay\_june and pay\_may since they were not statistically significant*

```
fit_glm_cv3 <- glm(Default ~ Limit_Bal+Education+Marriage+Age+Pay_Sep+Pay_Aug+Pay_July+
      Pay_May+Bill_Amt_Sep+Pay_Amt_Sep+Pay_Amt_Aug
      +Pay_Amt_July+Pay_Amt_June,
      data = train_set, family = binomial())
```

```
summary(fit_glm_cv3)
```

```
##
## Call:
## glm(formula = Default ~ Limit_Bal + Education + Marriage + Age +
##      Pay_Sep + Pay_Aug + Pay_July + Pay_May + Bill_Amt_Sep + Pay_Amt_Sep +
##      Pay_Amt_Aug + Pay_Amt_July + Pay_Amt_June, family = binomial(),
##      data = train_set)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -8.290e-01  1.182e-01  -7.012 2.34e-12 ***
## Limit_Bal    -7.376e-07  1.777e-07  -4.152 3.30e-05 ***
## Education    -1.141e-01  2.412e-02  -4.732 2.22e-06 ***
## Marriage      -1.891e-01  3.633e-02  -5.204 1.95e-07 ***
## Age           8.102e-03  2.023e-03   4.006 6.18e-05 ***
## Pay_Sep       5.768e-01  2.038e-02  28.299 < 2e-16 ***
## Pay_Aug       8.320e-02  2.318e-02   3.589 0.000332 ***
## Pay_July      7.434e-02  2.311e-02   3.217 0.001294 **
## Pay_May       6.916e-02  2.018e-02   3.428 0.000609 ***
## Bill_Amt_Sep -1.468e-06  3.020e-07  -4.861 1.17e-06 ***
## Pay_Amt_Sep  -1.063e-05  2.346e-06  -4.530 5.89e-06 ***
## Pay_Amt_Aug  -9.974e-06  2.186e-06  -4.564 5.02e-06 ***
## Pay_Amt_July -3.254e-06  1.729e-06  -1.882 0.059821 .
## Pay_Amt_June -3.922e-06  1.762e-06  -2.226 0.026002 *
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 23800  on 22476  degrees of freedom
## Residual deviance: 21044  on 22463  degrees of freedom
## AIC: 21072
##
## Number of Fisher Scoring iterations: 6
```

```
stepAIC(fit_glm_cv3)
```

```
## Start:  AIC=21072.31
## Default ~ Limit_Bal + Education + Marriage + Age + Pay_Sep +
##      Pay_Aug + Pay_July + Pay_May + Bill_Amt_Sep + Pay_Amt_Sep +
##      Pay_Amt_Aug + Pay_Amt_July + Pay_Amt_June
##
##              Df Deviance   AIC
## <none>                21044 21072
## - Pay_Amt_July  1      21048 21074
## - Pay_Amt_June  1      21050 21076
## - Pay_July      1      21055 21081
## - Pay_May       1      21056 21082
## - Pay_Aug       1      21057 21083
## - Age           1      21060 21086
## - Limit_Bal     1      21062 21088
## - Education     1      21067 21093
## - Bill_Amt_Sep  1      21069 21095
## - Marriage       1      21072 21098
## - Pay_Amt_Sep   1      21073 21099
## - Pay_Amt_Aug   1      21074 21100
## - Pay_Sep       1      21838 21864
##
## Call:  glm(formula = Default ~ Limit_Bal + Education + Marriage + Age +
##      Pay_Sep + Pay_Aug + Pay_July + Pay_May + Bill_Amt_Sep + Pay_Amt_Sep +
##      Pay_Amt_Aug + Pay_Amt_July + Pay_Amt_June, family = binomial(),
##      data = train_set)
##
## Coefficients:
## (Intercept)      Limit_Bal      Education      Marriage      Age
## -8.290e-01    -7.376e-07    -1.141e-01    -1.891e-01     8.102e-03
##      Pay_Sep      Pay_Aug      Pay_July      Pay_May  Bill_Amt_Sep
##  5.768e-01     8.320e-02     7.434e-02     6.916e-02    -1.468e-06
## Pay_Amt_Sep  Pay_Amt_Aug  Pay_Amt_July  Pay_Amt_June
## -1.063e-05    -9.974e-06    -3.254e-06    -3.922e-06
##
## Degrees of Freedom: 22476 Total (i.e. Null);  22463 Residual
## Null Deviance:      23800
## Residual Deviance: 21040      AIC: 21070
```

```
#VIF of the 3rd logistic regression model. VIF scores appear good.
vif(fit_glm_cv3)
```

```
##      Limit_Bal      Education      Marriage      Age      Pay_Sep      Pay_Aug
##      1.483357      1.135733      1.223487      1.251707      1.509065      2.626893
```

```
##      Pay_July      Pay_May Bill_Amt_Sep Pay_Amt_Sep Pay_Amt_Aug Pay_Amt_July
##      2.602541      1.817958      1.347766      1.162570      1.129421      1.097237
## Pay_Amt_June
##      1.080306
```

Four variables are discarded due to either statistical insignificance in the model or a high VIF value, indicating high multicollinearity between multiple variables, leading to 13 variables being kept.

```
pred_probs <- predict.glm(fit_glm_cv3, newdata = test_set, type = "response")
#Displays the predictions for a few values.
head(pred_probs)
```

```
##          2          4          5          8          11          16
## 0.1487036 0.2493531 0.1256277 0.1809134 0.2116167 0.2801776
```

```
#Sorts predictions into their respective class (0 or 1) depending on their value.
pred <- ifelse(pred_probs<0.5, 0,1)
#Creates and displays the confusion matrix table based on the actual and predicted values.
confusion_table <- table(test_set$Default, pred)
confusion_table
```

```
##      pred
##      0      1
## 0 5713  135
## 1 1243  397
```

```
#Creates the confusion matrix statistics for the logistic regression model.
cm_log <- confusionMatrix(confusion_table, positive = '1', mode = "everything")
#Saves the accuracy, precision, and recall values.
log_accuracy = accuracy(test_set$Default, pred)
log_precision = cm_log$byClass['Precision']
log_recall = cm_log$byClass['Recall']
log_pos_precision = cm_log$byClass['Neg Pred Value']
#Prints the accuracy, precision, and recall values.
print(paste("Accuracy: ", round(log_accuracy,3)))
```

```
## [1] "Accuracy:  0.816"
```

```
print(paste("Precision: ", round(log_precision,3)))
```

```
## [1] "Precision:  0.242"
```

```
print(paste("Recall: ", round(log_recall,3)))
```

```
## [1] "Recall:  0.746"
```

```
print(paste("Default Precision: ", round(log_pos_precision,3)))
```

```
## [1] "Default Precision:  0.977"
```

Once the model was run, the accuracy, precision, and recall were found for the prediction model. Accuracy describes how often the model is correct in its overall prediction. Precision identifies how often the model identifies those who default on their credit card out of all who do so, while recall identifies how often the model correctly identifies those who default on their credit card. Another way of describing precision and recall is precision is a measure of quality, while recall is a measure of quantity.

In the case of the logistic regression model, the accuracy was 0.816%, precision was 0.242%, and recall was 0.746%. The precision for predicting actual default cases correctly was 0.977%. Overall, the logistic regression model performed well for predicting whether a client would default even though the model contained a lot of false positives by predicting a client would default when in reality they would not. In the case of credit

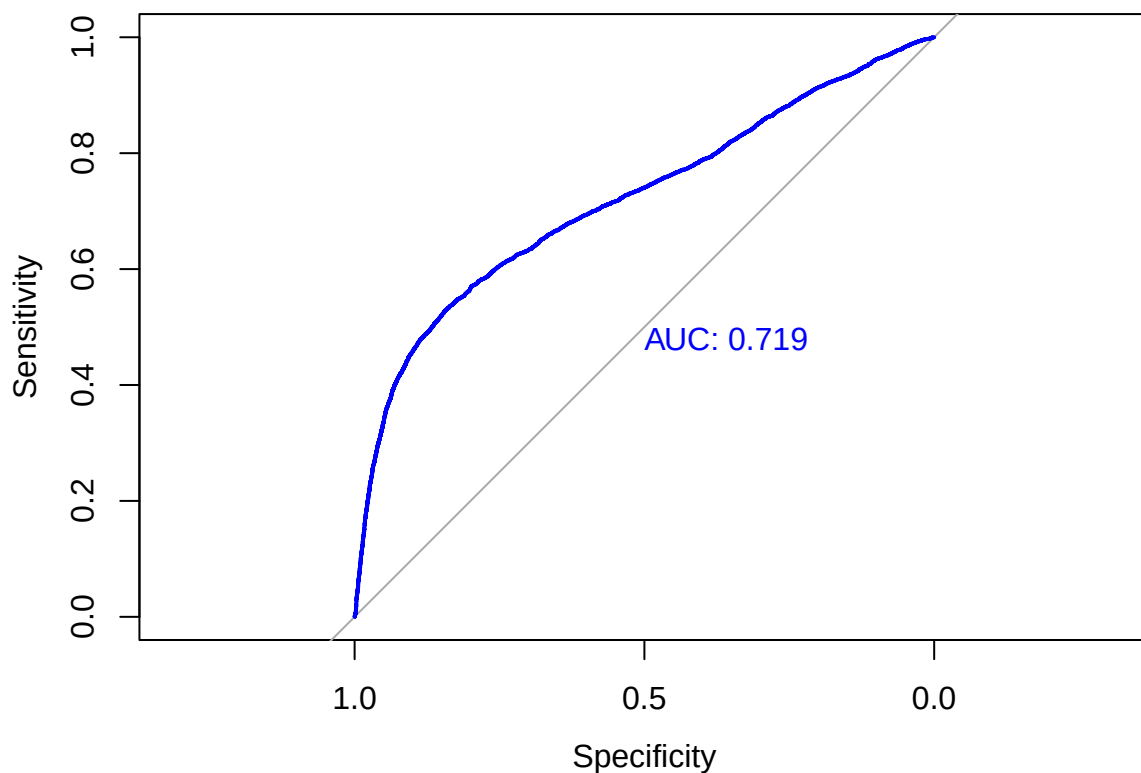
risk, it is best to have a higher recall rate compared to precision since it is better to have false positives occur compared to false negatives as they cost less money for the company. A false positive leads to higher operational costs for the company (sending payment reminders, lowering credit limit if applicable) while a false negative could cost the company thousands more if a client actually defaults on their payment.

The regression model's accuracy, specificity, and sensitivity can be improved by optimizing the cutoff point for the model. One way of doing so is using the Receiver Operating Characteristic (ROC) curve, which plots the true positive (sensitivity) against the false positive rate against various thresholds. The AUC curve can be used to measure the performance of the model, with a higher AUC number demonstrating better model performance. An AUC 0.8 and above indicates good model performance. The model has an AUC score of 0.719, which indicates acceptable model performance. This will be a factor to keep in mind for potential model improvements.

```
prob <- predict(fit_glm_cv3, type = "response")
# Create ROC curve
roc_obj <- roc(train_set$Default, prob, plot = TRUE, col = "blue", print.auc = TRUE)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```



```
# Find optimal cutoff (maximizes sensitivity + specificity)
#AUC performance = 0.718
opt <- coords(roc_obj, "best", ret = c("threshold", "sensitivity", "specificity"))
print(opt)
```

```
## threshold sensitivity specificity
## 1 0.2606347 0.5288577 0.8437697
```

```
#threshold = 0.251
#sensitivity = 0.548
#specificity = 0.822
```

The ROC curve supplies various cutoff values for the logistic regression model. From the ROC curve, the optimal cutoff point to maximize all three measurements (accuracy, specificity, and sensitivity) is 0.251, while the cutoffs to maximize sensitivity and specificity are 0.548 and 0.822, respectively. The optimal cutoff from the AUC curve, which maximizes both sensitivity and specificity, is 0.719. There is a tradeoff for each cutoff point, so it is up to the user to determine which is best for the purpose of the model. I chose to use 0.548 since I want to maximize both the overall accuracy of the model while accurately identifying those who will default on their credit card.

```
pred_new <- ifelse(pred_probs<0.529, 0,1)
#Creates and displays the confusion matrix table based on the actual and predicted values.
confusion_table_new <- table(test_set$Default, pred_new)
confusion_table_new
```

```
##      pred_new
##      0      1
## 0 5751    97
## 1 1315   325
```

```
#Creates the confusion matrix statistics for the logistic regression model.
cm_log_opt <- confusionMatrix(confusion_table_new, positive = '1', mode = "everything")
#Saves the accuracy, precision, and recall values.
log_accuracy_opt = accuracy(test_set$Default, pred_new)
log_precision_opt = cm_log_opt$byClass['Precision']
log_recall_opt = cm_log_opt$byClass['Recall']
log_pos_precision_opt = cm_log_opt$byClass['Neg Pred Value']
#Prints the accuracy, precision, and recall values.
print(paste("Accuracy: ", round(log_accuracy_opt,3)))
```

```
## [1] "Accuracy:  0.811"
```

```
print(paste("Precision: ", round(log_precision_opt,3)))
```

```
## [1] "Precision:  0.198"
```

```
print(paste("Recall: ", round(log_recall_opt,3)))
```

```
## [1] "Recall:  0.77"
```

```
print(paste("Default Precision: ", round(log_pos_precision_opt,3)))
```

```
## [1] "Default Precision:  0.983"
```

From the results, the recall, and default precision improved with the new threshold value, while accuracy and precision had decreased. Using this new threshold helps predict a default more accurately, but does so by incorrectly labeling someone as ‘default’ when they would not do so more often than the previous threshold mark. Overall, this model is kept since the overall accuracy improves and it’s more important to identify those who will default.

## Random Forest

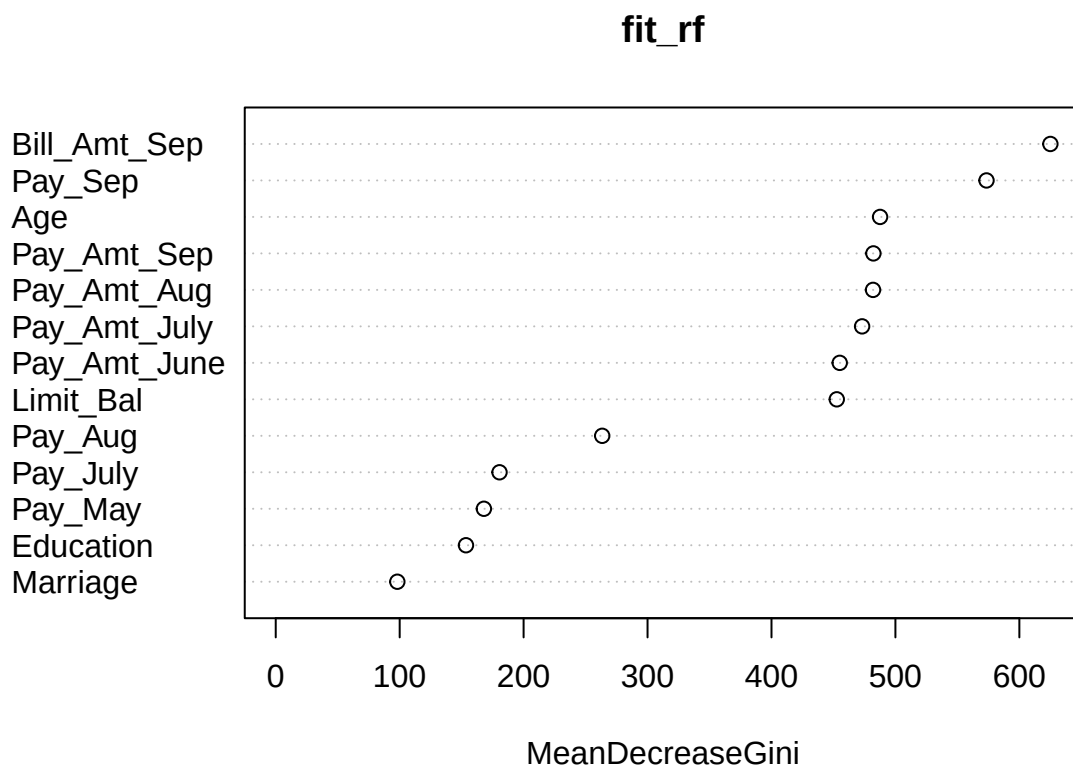
Another prediction model used is random forest. Random forest is a classifying method consisting of many decision trees. By creating a “forest” of decision trees, the classifying model hopes to select it’s best model by running many different decision trees and “takes the majority” to determine classification. To do so, random forest uses out-of-bag sampling.

A random forest model is created to determine the probability of credit card default:

```
set.seed(123)

n_defaults <- sum(train_set$Default == 1)

#Random Forest for variables. mtry = 4 since there are 13 variables (square root of 11 is close to 4).
fit_rf <- randomForest(factor(Default) ~Limit_Bal+Education+Marriage+Age+Pay_Sep+Pay_Aug+Pay_July+
  Pay_May+Bill_Amt_Sep+Pay_Amt_Sep+Pay_Amt_Aug
  +Pay_Amt_July+Pay_Amt_June,
  mtry = 4, ntree = 1000, sampsize = c(n_defaults, n_defaults),
  strata = factor(train_set$Default), data = train_set)
varImpPlot(fit_rf)
```



```
#Predicts values in the test set.
predict_rf <- predict(fit_rf, test_set)
#Creates the confusion matrix table for the random forest model.
confusion_table_rf <- table(test_set$Default, predict_rf)
#Creates and displays the confusion matrix statistics for the random forest model.
cm_rf <- confusionMatrix(confusion_table_rf, positive = '1', mode = "everything")
cm_rf
```

```
## Confusion Matrix and Statistics
##
##      predict_rf
##      0      1
## 0 4969  879
```



```
## 1 680 960
##
## Accuracy : 0.7918
## 95% CI : (0.7824, 0.8009)
## No Information Rate : 0.7544
## P-Value [Acc > NIR] : 1.079e-14
##
## Kappa : 0.4169
##
## McNemar's Test P-Value : 5.313e-07
##
## Sensitivity : 0.5220
## Specificity : 0.8796
## Pos Pred Value : 0.5854
## Neg Pred Value : 0.8497
## Precision : 0.5854
## Recall : 0.5220
## F1 : 0.5519
## Prevalence : 0.2456
## Detection Rate : 0.1282
## Detection Prevalence : 0.2190
## Balanced Accuracy : 0.7008
##
## 'Positive' Class : 1
##
```

```
#Saves the accuracy, precision, and recall values.
rf_accuracy = accuracy(test_set$Default, predict_rf)
rf_precision = cm_rf$byClass['Precision']
rf_recall = cm_rf$byClass['Recall']
rf_pos_precision = cm_rf$byClass['Pos Pred Value']
#Prints the accuracy, total precision, recall, and default precision values.
print(paste("Accuracy: ", round(rf_accuracy,3)))
```

```
## [1] "Accuracy: 0.792"
```

```
print(paste("Precision: ", round(rf_precision,3)))
```

```
## [1] "Precision: 0.585"
```

```
print(paste("Recall: ", round(rf_recall,3)))
```

```
## [1] "Recall: 0.522"
```

```
print(paste("Default Precision: ", round(rf_pos_precision,3)))
```

```
## [1] "Default Precision: 0.585"
```

From the input printed and the plot provided, it is seen that the pay September variable and bill amount in September, as well as age are important variables in determining credit card default. It can also be argued pay amount in September, pay amount in August and pay amount in July are important variables in determining credit card default.

Looking at the confusion matrix, the random forest model's accuracy was 0.792%, precision was 0.585%, and recall was 0.522%. The precision for predicting actual default cases correctly was 0.585%. Overall, the random forest model was slightly better in its accuracy and much better for its precision. However, the logistic regression model performed better than the random forest model when predicting actual default cases, as evidenced by the recall and default precision values.

## Conclusion

In this project, logistic regression and random forest models were created to predict if an individual would default on their credit card.

First, the data was cleaned for accuracy and manipulated to view distributions and trends in the data. From the tables and plots created, the data had more females than males and was skewed in age, with participants below the age of 40 much more prevalent than participants over the age of 40.

Next, prediction models were created to predict an individual's chances of defaulting on their credit card. The first model used was a logistic regression model. This model was used to predict if an individual would default on their credit card based on their information. This model is great for predicting a Yes/No classification for individuals. From the model created, three individuals were created with their unique information. In the example above, all three individuals created had a good chance of not defaulting on their credit card.

A random forest model was also created to determine the most important variables in a prediction model, as well as to see the accuracy of the created model. From the results, the random forest model could accurately predict someone not defaulting on their credit card, but had a more difficult time accurately predicting when someone would default on their credit card.

When comparing the two models, the following table was created:

```
set.caption("Performance for Logistic Regression and Random Forest Models")
data.table = rbind(c(log_accuracy_opt, log_precision_opt, log_recall_opt, log_pos_precision_opt), c(rf_
colnames(data.table) = c("Accuracy", "Precision", "Recall", "Default Precision")
rownames(data.table) = c("Logistic Regression", "Random Forest")

pander(data.table)
```

Table 1: Performance for Logistic Regression and Random Forest Models

|                            | Accuracy | Precision | Recall | Default Precision |
|----------------------------|----------|-----------|--------|-------------------|
| <b>Logistic Regression</b> | 0.8114   | 0.1982    | 0.7701 | 0.9834            |
| <b>Random Forest</b>       | 0.7918   | 0.5854    | 0.522  | 0.5854            |

Overall, it seems the logistic regression model has a higher recall and default precision rate than the random forest model, but does worse than the random forest model in accuracy and precision. This means the logistic regression model has less false negatives than the random forest model, but also has more false positives. Though the accuracy seems fairly high for the random forest prediction model, I am concerned with the false negative rates and the low default precision percentage.

Though these prediction models are acceptable, there is room for improvement, particularly in accurately predicting client that will default. I believe adding certain variables such as credit score, credit age, and credit card utilization can help improve the prediction models.

Thank you for viewing my project.

END